Journal of Machine Learning Research 1 (2026) 1–5 Submitted 7/26; Published

kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

Moussa Kassim[1,2], **Martin Ocharo**[1,2], **Alessia Vignoli**[3,4], **Leonardo Tenori**[3,4], **and Stefano Cacciatore**[1,2] STEFANO.CACCIATORE@ICGEB.ORG

[1] *Bioinformatics Unit, International Center for Genetic Engineering and Biotechnology, Cape Town 7925, South Africa*

[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa*

[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*

[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

Editor: To be assigned

Abstract

KODAMA searches for latent structure by maximizing the held-out predictability of an evolving label vector. We present `kodama-cpp`, a standalone C++17 implementation that preserves this objective while reorganizing its repeated numerical work for multicore CPU, NVIDIA CUDA, and Apple Metal. One typed core owns float32 data, folds, labels, classifier workspaces, and graph outputs. KODAMA optimization exposes two classifiers: KNN and SIMPLS followed by latent-space LDA. Backend-specific implementations provide nearest-neighbor search, k-means initialization, label-aware SIMPLS cross-products, reusable workspaces, and strict backend identity without linking R, Python, FAISS, cuVS, RAFT, or Armadillo. Independent optimization runs can start from different sampled landmarks, and every proposal cycle performs exactly one cross-validation evaluation. The public API supports data matrices, supplied neighbor graphs, graph construction, float32 randomized PCA, and UMAP/openTSNE visualization; thin R and Python wrappers call the same interface. Validation separates classifier parity, kernel runtime, complete KODAMA runtime, and external-label diagnostics so that internal cross-validated accuracy is not mistaken for ground-truth recovery.

Keywords: KODAMA, cross-validation, unsupervised learning, heterogeneous computing, manifold learning

1 Introduction

KODAMA treats a sample-label vector as an optimization variable rather than observed truth. A candidate labeling is useful when a classifier trained on some samples can reproduce it on held-out samples. The original method and R package established this cross-validated accuracy principle and used an ensemble of optimized label vectors to correct pairwise dissimilarities before visualization (Cacciatore et al., 2014, 2017). The method is therefore related to prediction-strength and stability validation, but differs by placing held-out predictability inside the label search itself (Tibshirani and Walther, 2005).

Repeated classifier fitting makes KODAMA a systems problem as well as a statistical one. Neighbor graphs, fold matrices, class encodings, projections, and evolving labels recur

across many proposal cycles. `kodama-cpp` moves these operations into a standalone float32 core, keeps the KODAMA state machine common across backends, and exposes the same contract to R and Python. The contribution is a new portable implementation of established mathematics, not a replacement learning objective.

2 KODAMA objective and search

For data $X \in \mathbb{R}^{n \times p}$, labels y, folds Π, and classifier family F, let

$$A(y; X, F, \Pi) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[y_i = \widehat{y}_i^{(-\Pi(i))}\right]. \tag{1}$$

KODAMA searches for label vectors with high A; it does not interpret A as external accuracy. The current implementation uses either KNN or SIMPLS plus LDA in the requested feasible latent dimension (de Jong, 1993). Fold assignments remain fixed within a run, and constrained samples move as one group.

Each of M runs uses seed $s + r$, selects up to L landmarks, and initializes `splitting` labels by k-means. When the requested $L \geq n$, the historical rule $\lceil 0.75n \rceil$ is retained. The default initial class count is 100 for $n < 40000$ and 300 otherwise. At cycle t, the previous held-out predictions propose group relabelings. The largest proposal size decreases smoothly from broad to local moves,

$$q_{\max}(t) = 1 + \left\lfloor (G-1)\left[1 - p_t^2(3 - 2p_t)\right] \right\rfloor, \qquad p_t = \frac{t+1}{T+1}, \tag{2}$$

and q groups are sampled uniformly from $1, \ldots, q_{\max}(t)$. Class-transition proposals can absorb one or more source labels whose held-out predictions preferentially transition into a target, without permitting a one-class result. PLS–LDA additionally applies transition-driven coarsening when fragmented classes are unstable.

The proposed vector receives exactly one new CV pass. Raw A is reported, while the default acceptance score guards against trivial predictability:

$$S(y) = A(y)\sqrt{1 - \sum_k p_k^2} - (1 - A(y))P(y), \tag{3}$$

where p_k is a class proportion and $P = 0$ for KNN; for PLS–LDA, P is the larger of normalized label entropy and an effective-class fragmentation term. A proposal improving the best S is retained. The current chain can also accept a worse proposal with probability $\exp[(S_{new} - S_{cur})/\tau_t]$, where $\tau_t = 0.10(1 - A_{cur})(1 - t/T)$. These grouped, guarded search dynamics extend the historical implementation but use only CV predictions, never reference labels.

After T cycles, labels are projected from landmarks to all samples. Across runs, an original graph edge (i, j) has agreement a_{ij} equal to the fraction of valid runs assigning its endpoints the same label. Its corrected distance is $d'_{ij} = (1 + d_{ij})/a_{ij}^2$; zero agreement removes the edge. This sparse graph is the KODAMA representation supplied to visualization.

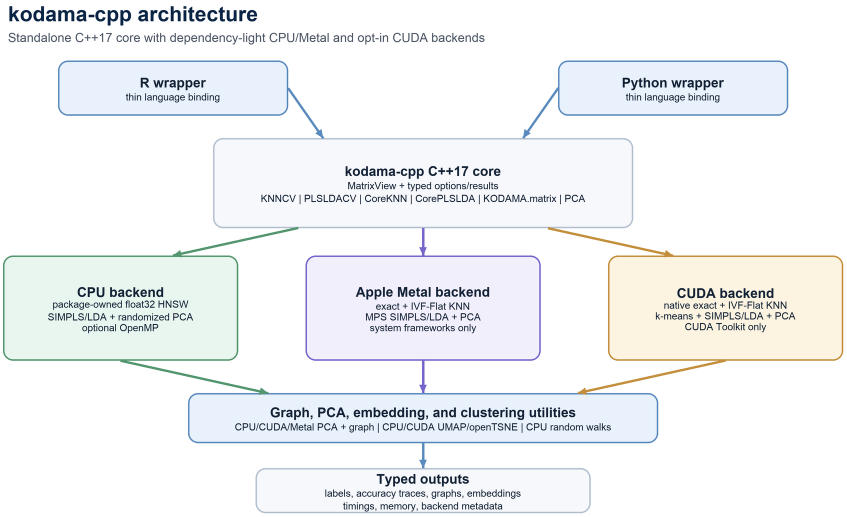

Figure 1: The C++17 core owns KODAMA state and exposes one typed API to R and Python. Backend-specific kernels share the same folds, proposals, component-count semantics, and result contract.

One independent run. Initialize landmarks, labels, folds, and one CV prediction; for $t = 1, \ldots, T$: propose grouped and class-transition moves from the current predictions; evaluate the proposal once by CV; update the best state by S and the current state by the cooling rule. Return the best raw accuracy and labels. Repeat independently for $r = 1, \ldots, M$, project labels, and reweight the shared graph by run-wise agreement.

3 Standalone heterogeneous implementation

Figure 1 shows the ownership boundary. CPU provides package-owned HNSW (Malkov and Yashunin, 2020), SIMPLS/LDA, PCA, graph operations, and UMAP/openTSNE. CUDA provides exact/IVF KNN, batched k-means, label-aware SIMPLS/LDA, PCA, and embedding kernels; Metal provides exact/IVF KNN, k-means, MPS-assisted SIMPLS/LDA, and PCA. Unavailable accelerators raise errors rather than silently executing CPU code.

Dense one-hot responses are avoided in PLS–LDA: $X^\top Y$ is computed from class-wise feature sums, and compact active labels are mapped to contiguous codes each cycle. Fold indices and workspaces are reused. The requested component count is evaluated wherever mathematically feasible; it is not selected by internal validation. Results include predictions, folds, confusion matrices, label ensembles, timings, memory, search parameters, and the backend that executed.

Graph input is an alternative API. KNN consumes supplied indices and distances; graph-only PLS–LDA uses one documented self-tuning normalized-Laplacian transform. PCA is an auxiliary primitive, not part of the KODAMA objective. Pinned fastEmbedR UMAP/openTSNE kernels use matched contracts for classic and corrected graphs, with

Table 1: Selected implementation validation. Accuracy is CPU/accelerator; runtime is seconds.

Platform	Dataset/kernel	CPU	Accel.	Speedup	Accuracy
CUDA	MNIST KNNCV	76.769	4.753	16.2	.974/.973
CUDA	MetRef PLSLDACV	3.196	0.118	27.2	.992/.992
CUDA	USPS PLSLDACV	3.760	0.158	23.8	.684/.687
Metal	MetRef KNNCV	11.145	0.026	425.0	.827/.827
Metal	MetRef PLSLDACV	3.395	1.054	3.2	.992/.992

direct float32 CSR construction and explicit binary or fuzzy UMAP edges (McInnes et al., 2018; van der Maaten and Hinton, 2008).

4 Validation and scope

Validation separates kernels from end-to-end runtime. CTest checks predictions, folds, requested components, PCA invariants, strict backend metadata, float32 inputs, graph utilities, and the frozen `0.1.0` API; wrappers test matrix, graph, PCA, and visualization calls. Table 1 reports within-platform measurements.

At matched MetRef KNN settings ($M = T = 100$, `splitting`=50), the legacy KNN-capable KODAMA R 2.4.1 took 610.5 seconds; kodama-cpp CPU1, CPU4, and CUDA took 965.3, 235.5, and 2.36 seconds. All reached best raw $A = 1$. This is neither a current-CRAN nor a trajectory-parity comparison.

An easily predicted labeling may still be too coarse, so raw accuracy, proposal score, and external diagnostics are separated. At $M = 100$, increasing T from 20 to 100 improved median accuracy and reduced fragmentation on USPS and MetRef PLS–LDA; MetRef KNN remained over-coarsened. M instead controls ensemble precision: worst-case agreement-edge standard error is 0.05 at $M = 100$, and all four $M = 50$ graphs correlated at least 0.9888 with their $M = 100$ counterparts. ARI was nonmonotone and was not tuned. A five-dataset panel, including 44,808 samples in its largest matrix, showed classifier- and dataset-dependent silhouette changes; the supplement retains all positive and negative results.

5 Availability and limitations

Version `0.1.0` is MIT licensed at `https://github.com/tkcaccia/kodama-cpp`; adapted portions retain upstream notices and the Metal Apache-2.0 exception. Provenance pins source snapshots, and release tests compile-link the API. The final commit and DOI await archival deposition.

Repeated CV fitting remains the main cost, and M-run orchestration is not fully device-resident. Approximate search may change neighbor ties within documented recall tolerances. Graph-only PLS–LDA is a spectral surrogate, not equivalent to data-input PLS–LDA; strict backend metadata exposes these boundaries.

References

Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.

Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.

Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.